
OptimSolution

Contents

Overview

Optimsolution is a C++ optimization framework designed for repeatable, high-throughput experimentation with population-based metaheuristics and numerical optimizers on benchmark and application-driven objective functions. Its primary objective is to standardize execution (initialization, stopping rules, logging) so that results are comparable and fully reproducible across methods, problems, and runs.

The project supports two primary usage modes: a command-line interface (CLI) for batch execution and automation, and a graphical user interface (GUI) for interactive method/problem selection, live log inspection, and convergence visualization via CSV output.

What this manual covers

This manual provides step-by-step installation and build instructions for Windows and Linux (Debug and Release), covers both CLI and GUI usage, explains the configuration model (optimsolution.cfg, the [global] section, and runtime overrides), and describes the expected output artifacts (run summaries, CSV files, and convergence traces).

Core capabilities

- Unified interface for running methods on problems, with consistent logging and a standardized end-of-run summary.
- Explicit termination budgets (function evaluations and/or iterations), suitable for benchmarking protocols.
- Reproducibility support through independent runs and consistent initialization/stopping conventions.
- Configuration via optimsolution.cfg, including shared defaults under [global] and per-run overrides.
- CSV export and convergence recording to enable plotting and cross-run comparisons.
- GUI with separate method/problem dropdowns, Run/Stop process control, single-line log entries, and per-run output tabs.

Terminology and key concepts

The following terms are used consistently throughout this manual. Maintaining strict terminology is important for correct interpretation of settings and output files.

- Method: The algorithm/optimizer being executed (e.g., DE variants, CMA-ES, etc.).
- Problem: The objective function/benchmark with defined bounds and dimension.
- Run: One independent execution of a method on a problem under a specific budget and configuration.
- Budget: The computational limit (evaluations/iterations) that determines when a run terminates.
- CSV / Convergence: Output files capturing best-so-far progress during the run.
- Run Summary: The standardized end-of-run report (e.g., best/mean, timing, success criteria when defined).

What to expect next

The next chapters provide concrete build commands, practical guidance for separating CLI and GUI settings, and execution examples. Instructions are written to support clean builds and reproducible runs from any working directory without ambiguity.

System Requirements

optimsolution is a C++ project built with CMake. It can be used as a console application (CLI) and, optionally, as a Qt6-based graphical application (GUI). Requirements differ depending on whether you build only the CLI or both CLI and GUI.

Supported Platforms

- Windows 10/11 (x64) with a Visual Studio / MSVC toolchain.
- Linux (x64) with GCC or Clang.

The codebase targets a 64-bit environment. Exact package names and availability on Linux depend on the distribution and the selected toolchain.

Dependencies

Required (CLI and GUI)

- CMake.
- A C++ compiler with C++17 support

Optional (GUI only)

- Qt6 (Core, Widgets, and any modules required by the GUI target).
- A Qt kit that matches the compiler toolchain (e.g., an MSVC kit on Windows).

On Windows, the most common build failures are caused by a mismatch between the compiler and the Qt build (e.g., MSVC vs MinGW). On Linux, a frequent cause is an incorrect or undiscoverable Qt prefix path.

Windows Installation and Build

This chapter describes the Windows 10/11 build workflow using MSVC and CMake. Instructions are organized by CLI and GUI, and by Debug and Release configurations. The GUI requires Qt6 and a correct Qt prefix path.

Prerequisites

- Visual Studio 2022 with the “Desktop development with C++” workload installed.
- CMake available on PATH (or via Visual Studio).
- A Qt6 MSVC kit (e.g., C:\Qt\6.10.1\msvc2022_64) to build the GUI.

It is recommended to run commands from “Developer PowerShell for VS 2022” so the MSVC environment is correctly initialized.

Console (CLI) – Debug

Clean configure/build (out-of-source):

```
Remove-Item -Recurse -Force .\build -ErrorAction SilentlyContinue
cmake -S . -B build -G "Visual Studio 17 2022" -A x64
cmake --build build --config Debug -j
.\build\Debug\optimsolution_cli.exe --help
```

With a multi-config generator (Visual Studio), Debug/Release selection happens at build time via --config. CMAKE_BUILD_TYPE is not used in this scenario.

Console (CLI) – Release

```
cmake --build build --config Release -j
.\build\Release\optimsolution_cli.exe --help
```

You may reuse the same build directory for Debug and Release. If you change generator/toolchain, perform a clean build.

Graphical User Interface (GUI) – Debug

The GUI target is enabled through Qt6. Two common integration patterns are used: (a) specifying CMAKE_PREFIX_PATH at configure time, or (b) using CMAKE_PROJECT_INCLUDE to load cmake/optimsolution_gui.cmake.

Pattern A: CMAKE_PREFIX_PATH

```
Remove-Item -Recurse -Force .\build -ErrorAction SilentlyContinue
cmake -S . -B build -G "Visual Studio 17 2022" -A x64 -
DCMAKE_PREFIX_PATH="C:\Qt\6.10.1\msvc2022_64"
cmake --build build --config Debug -j
.\build\Debug\optimsolution_gui.exe
```

Pattern B: CMAKE_PROJECT_INCLUDE (GUI include)

If the repository is structured to enable GUI integration via a separate CMake include, pass the include file at configure time. This keeps GUI-related CMake logic encapsulated.

```
Remove-Item -Recurse -Force .\build -ErrorAction SilentlyContinue
cmake -S . -B build -G "Visual Studio 17 2022" -A x64 -
DCMAKE_PROJECT_INCLUDE=cmake/optimsolution_gui.cmake -
DCMAKE_PREFIX_PATH="C:\Qt\6.10.1\msvc2022_64"
cmake --build build --config Debug -j
.\build\Debug\optimsolution_gui.exe
```

Graphical User Interface (GUI) – Release

```
cmake --build build --config Release -j
.\build\Release\optimsolution_gui.exe
```

Common Issues and Fixes (MSVC/Qt)

Qt kit mismatch (MSVC vs MinGW)

Ensure the Qt installation used by CMake is built for MSVC (e.g., msvc2022_64). Otherwise, link errors or binary incompatibilities are expected.

CMake cannot find Qt

- Verify -DCMAKE_PREFIX_PATH points to the correct Qt prefix.
- Alternatively, set Qt_DIR to a path containing the Qt6*Config.cmake files.
- Perform a clean configure when changing Qt paths.

'identifier not found' errors for Qt classes

These errors typically indicate missing headers or the use of API calls that do not match the selected widget class.

- If you use QPlainTextEdit, ensure <QPlainTextEdit> is included.
- If you use QTextEdit and need to limit log lines, use textEdit->document()->setMaximumBlockCount(N) rather than textEdit->setMaximumBlockCount(...), which is not available on QTextEdit.

GUI launches but Qt DLLs are missing

Debug/Release builds require the corresponding Qt runtime DLLs. If you run outside Qt Creator, use windeployqt or ensure the Qt bin directories are on PATH. Proper deployment is typically handled via packaging when distributing binaries.

General recommendation when changing build settings

- If you change generator, toolchain, or Qt prefix, delete build/ and reconfigure from scratch.
- Use separate build directories if you need multiple Qt kits or parallel configurations.

Linux Installation and Build

This chapter describes the Linux build workflow using CMake and GCC/Clang. The build is typically single-config (CMAKE_BUILD_TYPE), and out-of-source builds into a dedicated build directory are recommended.

Prerequisites

- CMake available on PATH.
- GCC or Clang with C++17 support.
- For the GUI: Qt6 (Core/Widgets) installed and discoverable by CMake.

Package names vary by distribution. If you rely on system packages, ensure the corresponding development packages are installed for Qt6 and any other required dependencies.

Console (CLI) – Debug

With single-config generators, Debug/Release is set at configure time using CMAKE_BUILD_TYPE.

```
rm -rf build
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build -j
./build/optimsolution_cli --help
```

If your target name or executable path differs, use the build output or list the build/ directory to locate the produced binary.

Console (CLI) – Release

```
rm -rf build
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
./build/optimsolution_cli --help
```

For benchmarking, a Release build is recommended. Debug builds are primarily used for development and debugging.

Graphical User Interface (GUI)

The GUI requires Qt6. On Linux, CMake typically discovers Qt6 either (a) via system packages that provide CMake config files, or (b) via a Qt installer, in which case you set CMAKE_PREFIX_PATH at configure time.

Example configure using CMAKE_PREFIX_PATH

```
rm -rf build
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -
DCMAKE_PREFIX_PATH=/path/to/Qt/6.x/gcc_64
cmake --build build -j
./build/optimsolution_gui
```

For detailed GUI guidance (layout, tabs, logging, and convergence plotting), refer to optimsolution_manual.pdf. For a visual reference of the GUI, use the figure optimsolution_gui.png.

Common Issues and Fixes (toolchain/Qt)

CMake cannot find Qt6

- Confirm Qt6 development packages are installed (distribution-specific names).
- If using an installer-based Qt, ensure CMAKE_PREFIX_PATH points to the correct Qt prefix.
- Delete build/ and reconfigure when switching Qt paths or toolchains.

Link errors or missing shared libraries at runtime

If the executable builds but fails to run due to missing shared libraries, use ldd to inspect dependencies and ensure the runtime environment (LD_LIBRARY_PATH / rpath) is correctly configured.

Switching compilers

If you switch from GCC to Clang (or vice versa), prefer a clean build in a new build directory to avoid mixing artifacts.

CLI Usage

The optimsolution CLI is designed for batch execution and automation. It enables running a selected method on a selected problem under an explicit budget with consistent stopping behavior, producing standardized logs and an end-of-run summary.

Basic syntax

Always use the built-in help to confirm the available syntax in your build.

```
optimsolution_cli --help
```

Selecting method and problem

Typical workflow: set method, problem, dimension (when applicable).

```
optimsolution_cli --method <METHOD> --problem <PROBLEM> --dim <D>
optimsolution_cli --method <METHOD> --problem <PROBLEM> --dim <D>
```

For fixed-dimension problems, the CLI may ignore --dim or provide an alternative mode.

CSV and convergence output (csv_enable/csv_convergence)

- csv_enable: enables/disables CSV creation.
- csv_convergence: enables convergence recording (best-so-far).

The GUI uses these CSV files to render convergence plots.

Run summary and relationship to configuration

Each run ends with a standardized run summary. Most parameters are read from optimsolution.cfg

Settings and Configuration Files

optimsolution relies on explicit configuration to support reproducibility and standardized execution. The primary configuration source is optimsolution.cfg located at the repository root. This manual follows a layered approach: shared defaults live in [global], while specialization is applied through dedicated sections and/or runtime overrides.

optimsolution.cfg

optimsolution.cfg is an INI-style configuration file with sections. It typically includes general runtime settings, method/problem parameters (where applicable), and output controls (logs/CSV).

Illustrative structure:

```
[global]
max_evals           = 200000
csv_enable          = 1
csv_convergence     = 1
output_root         = outputs

[JSO]
population          = 100
archive_size        = 200
```

Exact key/section names depend on the implementation. The example communicates the intent: shared defaults in [global], specialized values per method/problem when needed, and a separate section for GUI state.

[global] and runtime overrides

The [global] section acts as a default layer. More specific sections (e.g., method-specific) and runtime overrides take precedence over global values. This allows you to keep a stable baseline while changing only what is required for a particular run.

- Layer 1: [global] defaults.
- Layer 2: section overrides (e.g., method/problem sections).
- Layer 3: runtime overrides (parameters provided on the command line or set in the GUI before starting a run).

Runtime overrides are useful for parameter sensitivity studies and quick experiments without permanently editing the cfg. A critical requirement is that overrides must be recorded alongside the run to ensure full reproducibility.

Per-run temporary configuration snapshot

To ensure each run is fully documented, optimsolution creates a temporary snapshot of the effective configuration used for that run (global + overrides) and stores it together with the run outputs. This snapshot allows you to reproduce a run exactly even if the base optimsolution.cfg changes later.

Recommended practice:

- Store the snapshot cfg inside the run output directory (next to CSV and summary files).
- Log the snapshot path and the applied overrides in the run log.
- Use a unique run id or timestamp to avoid accidental overwrites.

Separating CLI settings vs GUI settings

The CLI and GUI share the same execution core settings (budgets, CSV, output paths, etc.). However, the GUI also needs user-experience state (e.g., last selected method/problem, layout preferences, plot visibility). To avoid confusion and to preserve experimental integrity, these settings should be clearly separated.

- CLI settings: anything that can affect numerical outcomes (e.g., budgets, seed, method parameters, CSV settings).
- GUI settings: UI state and presentation only (e.g., last selected method/problem, window geometry, plot options).

In practice, optimsolution.cfg may include a dedicated GUI section (e.g., [gui]) and/or the GUI may use a separate settings file. In all cases, the per-run snapshot should include only execution-relevant settings, not pure UI state, so the experimental footprint remains clean.

Minimal configuration for experiments

For typical benchmarking runs, keep the following in [global] and vary only what is strictly required per method:

```
[global]
max_evals       = 200000
csv_enable      = 1
csv_convergence = 1
output_root     = outputs
```

Then, define method-specific parameters in method sections so it is explicit what changes per method. Use runtime overrides only for ad-hoc changes and ensure they are captured in the per-run snapshot.

GUI Usage

The optimsolution GUI provides interactive method/problem execution with a focus on a clear workflow, live log inspection, and fast convergence visualization via CSV output. For a visual reference of the layout, see optimsolution_gui.png. For detailed GUI instructions, refer to optimsolution_manual.pdf.

Window layout and workflow

The standard workflow is: select a Method, select a Problem, adjust parameters/budgets where applicable, then start execution with Run. Each execution produces log output and opens an output tab named as method+problem.

Method/Problem selection

Selection is performed via two separate dropdowns. The method dropdown displays entries as “Full name (short)” to remain both readable and compact.

- Method dropdown: “Full name (short)”.
- Problem dropdown: a recognizable short/full name depending on the project.

Dimension handling (fixed vs variable)

For fixed-dimension problems, the GUI does not expose a dimension control and does not apply dimension auto-retry. For variable-dimension problems, a dimension field/control is shown when required.

- Fixed-dimension: hide the dimension control and avoid auto-retry.
- Variable-dimension: show the dimension control and pass it as a run parameter.

Run/Stop and process control

The execution button acts as a Run/Stop toggle. While running, Stop terminates the current process in a controlled manner to avoid orphan processes.

- Run: start an execution using the current configuration.
- Stop: terminate the current execution and return to idle state.

Log view

The log is presented as a sequence of single-line entries without blank lines. This improves readability and makes copying checkpoints (e.g., best values) straightforward without formatting noise.

Output tabs

Outputs are organized into multiple tabs. Each tab corresponds to a run (or logical execution) and is labeled using method+problem. Tabs are closable to keep the workspace clean.

Convergence plot from CSV

The convergence plot is built from CSV outputs (csv_enable/csv_convergence). The GUI reads the CSV files and renders best-so-far curves. The plot is designed to support comparing multiple problems under the same method within a single chart.

Axes and labels

- X-axis title: “Iterations”.
- Y-axis title: “Best solution”

Legend and overlay

The legend includes the problem short name and the dimension. The GUI supports overlaying multiple problems for the same method to compare curves in a single figure.

Export PNG

The plot can be exported as a 300 DPI PNG for reports and papers. Export should preserve axis titles and a readable legend.

Colored RUN SUMMARY styling

At the end of execution, the GUI renders a color-styled RUN SUMMARY to make key fields (e.g., best value, success, timing) immediately visible. This complements the raw log output and enables fast result inspection.

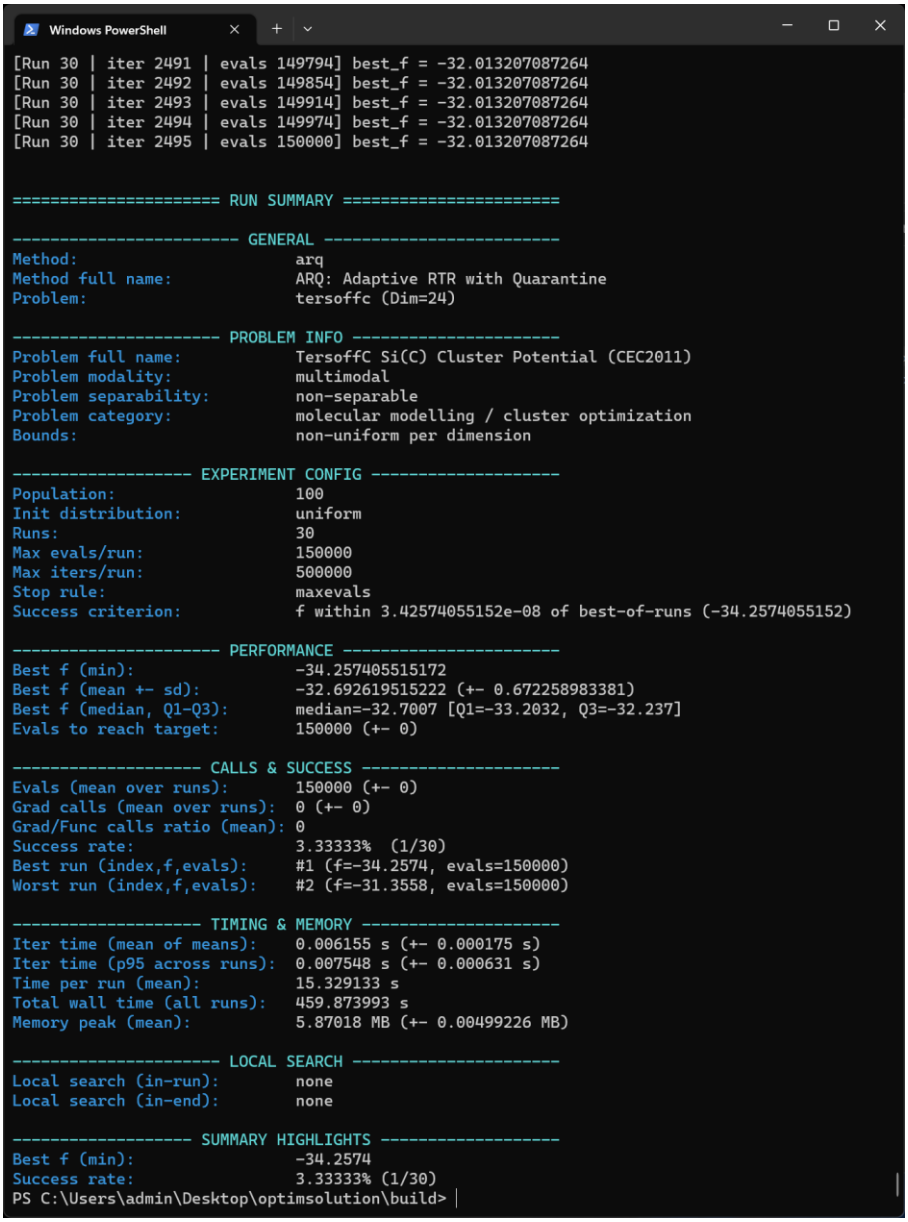
Examples

This chapter provides practical examples for running optimsolution via the CLI and the GUI, along with a minimal configuration baseline. Examples are intentionally template-style so they can be adapted to the actual method/problem names available in your build.

CLI run example

Pun a single execution

```
optimsolution_cli --method <METHOD> --problem <PROBLEM> --dim <D>
```



GUI run example

1) Launch the GUI executable (typically under build/Debug or build/Release on Windows).

2) Select a Method from the dropdown (displayed as “Full name (short)”).

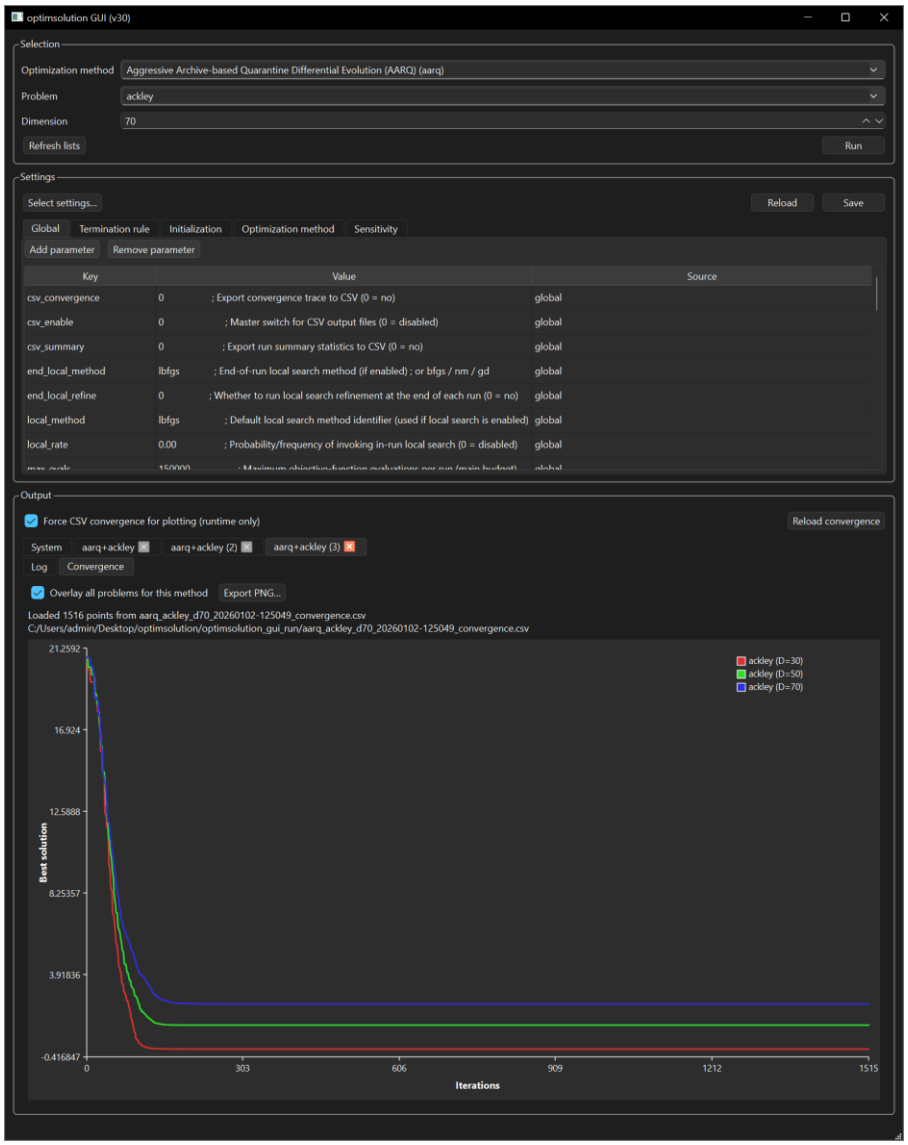
3) Select a Problem from the corresponding dropdown.

4) If the problem is variable-dimension, set the dimension. For fixed-dimension problems, the dimension control is hidden.

5) Click Run and monitor the log (single-line entries).

6) At completion, inspect the color-styled RUN SUMMARY and the output tab (method+problem).

7) If csv_enable/csv_convergence are enabled, view the convergence plot and export a 300 DPI PNG if needed.



Minimal configuration example (optimsolution.cfg)

The snippet below is a minimal, practical baseline for benchmarking. Adjust max_evals and output_root to match your workflow.

```
[global]
max_evals       = 200000
csv_enable      = 1
csv_convergence = 1
output_root     = outputs
```

Then add method sections for any method-specific parameters (e.g., population size) so it is explicit what changes per method.

Reproducibility checklist

- Always record budgets (max_evals/max_iters) and seed.
- Keep the per-run snapshot of the effective configuration in the output directory.
- Use a consistent output_root to avoid “lost” artifacts across different working directories.
- For curve comparisons, keep csv_convergence enabled and use a consistent logging frequency (if configurable).

Troubleshooting

This chapter consolidates common issues encountered during installation, build, or runtime usage of optimsolution and provides practical fixes. The goal is to diagnose root causes quickly (working directory, paths, Qt mismatches, CSV outputs) without introducing ad-hoc changes that harm reproducibility.

optimsolution.cfg not found from subfolders

Symptom: when running the CLI/GUI from build/Debug, build/Release, or another subfolder, the application fails to locate optimsolution.cfg even though the file exists at the repository root.

Practical fixes:

- Set the run working directory to the repository root (especially when launching from an IDE).
- Implement discovery logic that walks parent directories until optimsolution.cfg is found.
- Support an explicit option such as --config <path_to_cfg> to provide an absolute configuration path.
- Log the resolved configuration path so the issue is immediately visible in logs.

Note: this manual considers it desirable that the root cfg can be located regardless of the working directory, either through robust discovery or an explicit config path.

Qt build/link issues (Windows)

QPlainTextEdit: 'identifier not found'

If QPlainTextEdit is not recognized, a missing include or missing linkage to the correct Qt modules is a typical cause.

- Add the header include: <QPlainTextEdit>.
- Ensure the target links against Qt6::Widgets.
- Perform a clean build after changing modules or the Qt prefix.

QTextEdit: setMaximumBlockCount is not available

setMaximumBlockCount is an API of QPlainTextEdit. For QTextEdit, limit blocks via the document().

```
textEdit->document()->setMaximumBlockCount(1000);
```

Ensure <QTextEdit> and <QTextDocument> are included as required.

Qt kit mismatch (MSVC vs MinGW)

Use a Qt build that matches the compiler toolchain used to build the project. For MSVC builds, the Qt prefix must be an MSVC kit (e.g., msvc2022_64).

Qt build/link issues (Linux)

On Linux, common causes are missing Qt6 development packages or CMake failing to locate Qt because the prefix path is incorrect.

- Confirm Qt6 development packages are installed (distribution-specific names).

- If using an installer-based Qt, set a correct -DCMAKE_PREFIX_PATH at configure time.
- Delete build/ and reconfigure when switching Qt paths.

Missing or incorrect CSV / convergence output

Symptom: the GUI does not display a plot or shows an empty/incorrect figure even though a run completed.

Checks and fixes:

- Verify csv_enable=1 and csv_convergence=1 in the effective configuration (snapshot).
- Confirm the run output directory is what you expect (e.g., output_root).
- Ensure CSV files are created and contain data (not headers only).
- If plotting relies on naming conventions, ensure filenames match the expected pattern.

If the CSV is correct but the plot remains empty, confirm the GUI is reading the same output directory that the run wrote to, and that multiple working directories are not producing divergent outputs.

Run/Stop does not terminate execution correctly

Symptom: pressing Stop does not stop execution, or an orphaned process remains.

- Ensure the GUI holds the correct process handle and terminates the correct PID.
- Log state transitions (Run -> Stop) and the termination outcome.
- If child processes exist, ensure they are terminated as well (implementation-dependent).

General diagnosis strategy

- First, verify paths/working directory and the effective configuration snapshot.
- Then, verify toolchain/Qt mismatches and perform clean reconfigure after changing core build parameters.
- Finally, inspect outputs (CSV, logs) and confirm the GUI/CLI point to the same output_root.